

[Final Exam (Sample)]

Spring 2024: CSE 309: Theory of Automata

Instructors: Shahid Hussain, Imran Rauf

Sunday, May 26, 2024. Total Marks: 35 (final) + 5 (quiz) + bonus, Duration: 3 hours

Name: _____, Student ID: _____

Please read these instructions carefully:

- This exam contains 8 questions. Question 1 contributes towards the Quiz component of the course. The rest of the questions contribute towards the final exam component. Question 8 is a bonus question.
- Use of calculators, mobile, or any other communication devices is strictly prohibited for the entirety of the exam period.
- Please submit your devices in your bag at the front of the examination room.
- You may keep writing material with you on your desk.
- Read all questions carefully before you start answering and prioritize which questions to attempt first.
- You may ask for extra sheets for your rough work, scribbling, future planning etc., **anything on extra sheet(s) will not be graded.**
- Please write your name and student ID on this question paper clearly.
- Please do not use pencils or red/green/yellow pens for answers that you want to be graded.
- Instructor(s) will not entertain any queries during the exam.
- Please attach all extra sheets with your answer book.
- Please write clearly and legibly and avoid overwriting.
- Pay attention to the instructions for each question.
- Acquisition of answers through unfair means will automatically cancel your exam.
- Keep track of the time.
- **Your answer(s) will not be graded if unreadable (by the instructors).**
- **Your exam will not be graded if you don't follow these instructions.**

— Good Luck —

Question 1: Quiz 5 marks

Consider the following pair of languages:

$\text{DIAG} := \{ \langle M, k \rangle : M \text{ is a square matrix and sum of values on its diagonal is at most } k \}$

$\text{TSP} := \{ \langle M, k \rangle : M \text{ is a square matrix representing inter-city distances}$

and there is tour that visit all cities of total cost at most $k \}$

Note that TSP is a known **NP**-hard problem, whereas DIAG is in **P**. Which of the following must be true, assuming $\mathbf{P} \neq \mathbf{NP}$? [No justification is needed.]

- (a) F TSP $\in \mathbf{P}$
- (b) T DIAG is in **NP**
- (c) T There is a polynomial-time reduction from SAT to TSP.
- (d) T There is a polynomial-time reduction from DIAG to TSP.

Question 2: 6 marks

Chapter 1

- (a) (3 marks) Fix the alphabet $\Sigma = \{0, 1\}$. A run in a string $w \in \Sigma^*$ is a maximal substring of w consisting of all 0's or all 1's. For example, the string 001110011 has exactly four runs: $001110011 = 00 \cdot 111 \cdot 00 \cdot 11$.

Not sure
is completely
correct

Let L be the set of all strings in Σ^* that contains an even number of runs. For example, L contains 00011111110 but does not contain the string 10000111. Describe a regular expression for L .

Odd length runs mean that ϵ is not possible.

$$L = [(0 \cap (00)^+)^* \cap (1 \cap (11)^+)^*]^* \cup [(1 \cap (11)^+)^* \cap (0 \cap (00)^+)^*]^*$$

$\uparrow$ $\downarrow$ $\downarrow$ $\downarrow$
 $(0, 00, 0000, \dots)$ $(1, 11, 1111, \dots)$ $(1, 11, 1111, \dots)$ $(0, 00, 0000, \dots)$

- (b) (3 marks) Describe a CFG that generates the following language:

Chapter 2

$$L = \{a^x b^y c^z : y = x + z \text{ and } x, y, z \geq 0\}$$

$$x=2, z=3, y=5$$

pretty sure
mostly correct

$$S \rightarrow A \mid \epsilon \quad // x, y, z = 0$$

$$A \rightarrow A_2 C$$

$$A_2 \rightarrow a A_2 b \mid \epsilon$$

$$C \rightarrow b C c \mid \epsilon$$

$$S \rightarrow A$$

$$A \rightarrow A_2 C$$

$$= a A_2 b C$$

$$a a A_2 b b C$$

$$a a \quad b b C$$

$$= a a b b C$$

$$= a a b b b C c$$

$$= a a b b b b C c c$$

$$= a a b b b b b C c c c$$

$$= a a b b b b b b C c c c$$

$$= a a b b b b b b c c c$$

Question 3: 6 marks

Indicate whether following statements are **true** or **false** also justify your answer i.e., give a simple proof for a **true** statement and a counterexample for a **false** statement.

- (a) True Every regular language is context-free.

Chapter 2/2

It is
true, proof
is easy

It is possible to convert all DFAs to CFGs, and all regular languages can be represented by DFAs for each, likewise all context-free languages can be represented by CFGs.

Hence, every regular language is context-free.

- (b) _____ Following language L is **NP-hard**.

Chapter 7

is false
I think.

$$L = \{ \langle M \rangle : M \text{ is a DFA that accepts all strings} \}$$

$$L(M) = \Sigma^*$$

Question 4: Chapter 5 5 marks

Show that if A is recognizable and $A \leq_m \bar{A}$, then A is decidable.

pretty sure
is
completely
correct

If $A \leq_m \bar{A}$, then by definition of the reducibility function,
 $(\bar{A}) \leq_m (\bar{\bar{A}}) \rightarrow \bar{A} \leq_m A$.

Now, we are given that A is recognizable. From this,
it follows that $\bar{A}$ must be recognizable, as $\bar{A}$
reduces to A .

Hence, if both A and $\bar{A}$ are recognizable, then it
is known by a theorem that a problem is decidable
iff it and its complement are recognizable. Since this
is the case in this scenario, A is decidable.

Question 5: Chapter 7 6 marks
Show that following problems are in class NP.

- (a) **3SAT**: Given a CNF formula ϕ with n variables and m clauses, is there an assignment to the variables that satisfies all the clauses.

pretty sure
correct

We can show that $3SAT \in NP$ by creating a polynomial time verifier. Let that verifier be V . V takes in as input a CNF formula ϕ with n variables and m clauses, and a certificate c , which will include a number of assignments for variables. // c is a set of assignments.

$V =$ "on input $\langle \phi \rangle, c \rangle$:

- ① Check if variables of c match variables in ϕ .
- ② If check passed, then apply assignments to variables, else reject.
- ③ If ϕ evaluates to true, accept, else if it evals to false, reject.

- (b) **Dominating Set**: Given a graph $G = (V, E)$ and a number k , is there a set $V' \subseteq V$ such that $|V'| \leq k$ and for every vertex $v \in V$, either v is in V' or v is adjacent to a vertex in V' .

pretty sure
correct

Verifier (G, k, c)

We can prove that this problem $\in P$ by creating a polynomial time verifier for it. Let that verifier be V . V will take as its inputs $\langle G \rangle, k$, and a certificate c , which is a graph itself.

$V =$ "on input $\langle G \rangle, k, c \rangle$:-

- ① Verify that the vertices V' in c are less than k .

- ② If the check passed, then verify the following for every $v \in V$ (os graph $\langle G \rangle$):
 - is $v \in V'$, or
 - v is adjacent to a vertex in V' .

- ③ If both step 1 and step 2 is passed, then accept. Else, reject.

V runs in polynomial time, hence this problem $\in NP$ by the definition of the class of NP.

Question 6: **Chapter 7** 5 marks

If L is a language over some alphabet Σ^* then the complement of L denoted as $\bar{L}$ is defined as $\bar{L} = \Sigma^* - L$. We say a language L is in the class coNP if its complement $\bar{L}$ is the class NP . Prove/disprove that " $\text{NP} = \text{coNP}$ if and only if $A \leq_P B$ and $B \leq_P A$ where A is an NP -complete language and B is a coNP -complete language."

Pretty
sure
correct,
though
not
completed
cause
lazy

$L \in \text{NP}$ is $\bar{L} \in \text{NP}$

prove: $\text{NP} = \bar{\text{NP}}$ iff $A \leq_P B$ and $B \leq_P A$

- $A \in \text{NP-Complete}$
- $B \in \bar{\text{NP-Complete}}$

Proving Forward Direction:-

if $A \leq_P B$, and A is NP-Complete , then B must by definition also be NP-Complete .

However, we have $B \in \bar{\text{NP-Complete}}$, and $B \leq_P A$, so by this, $A \in \bar{\text{NP-Complete}}$.

Clearly, these two points are contradictory, unless $\text{NP} = \bar{\text{NP}}$, and this is the only conclusion that can be reached.

Proving Backward Direction:-

If we have $\text{NP} = \bar{\text{NP}}$, then for any problems A and $B \in \text{NP}$, A and $B \in \bar{\text{NP}}$. Hence, is $A \leq_P B$, and we consider A to be

Question 7:

Chapter 5

7 marks

Let $T = \{ \langle M \rangle \mid M \text{ is a TM that accepts } w^{\text{rev}} \text{ whenever it accepts } w \}$. Prove by reduction that T is undecidable.

Please ensure that you give a complete argument regarding why your reduction works. You can choose any undecidable language that we have seen in class for your reduction.

1022,
the
answer
is of
Anas.

Let us assume that T is decided by a TM H . We can reduce H to A_{TM} , hence $H \leq_m A_{\text{TM}}$. By the definition of the reducible function, A_{TM} will be decidable, since H is decidable. Let S be the TM that decides A_{TM} . To construct S , we can use TM H as part of the construction. TM S will take as input $\langle M, w \rangle$, where w is a string.

$S = \text{"on input } \langle M, w \rangle \text{"}$

① Construct a TM M' based on M and w :

$M' = \text{"on input } \langle M \rangle, \text{ a string:"}$

- If $w \neq w^{\text{rev}}$

Let $T = \{ \langle M \rangle \mid M \text{ is a TM that accepts } w \text{ whenever it accepts } w \}$.

Assume T is decidable and let decider R decide T . Reduce from A_{TM} by constructing a TM S as follows:

• S : on input $\langle M, w \rangle$

1. create a TM Q as follows:

On input x :

1. if x does not have the form 01 or 10 reject.

2. if x has the form 01 , then accept.

3. else (x has the form 10), Run M on w and accept if M accepts w .

2. Run R on

3. Accept if R accepts, reject if R rejects.

Because S decides A_{TM} , which is known to be undecidable, we then know that T is not decidable

Question 8: Bonus 5 marks

Show that the class **NP** is closed under the star operation. That if L is in **NP** then L^* is also in **NP**.

this
answer
is
correct
I
recheck

To show that $L^* \in \text{NP}$, we can create a polynomial time NTM M that takes as input a string $w \in L$. If $L^* \in \text{NP}$, then $w^* \in L^*$.

$M =$ "On input w :

1. If $w = \epsilon$ then accept.
2. Nondeterministically select a number m such that $1 \leq m \leq |w|$.
3. Nondeterministically split w into m pieces such that $w = w_1 w_2 \dots w_m$.
4. For all i , $1 \leq i \leq m$: run M_1 on w_i . If M_1 rejected then reject.
5. Else (M_1 accepted all w_i , $1 \leq i \leq m$), accept."

Observe that steps 1. and 2. take time $O(n)$, because the size of the number m is bounded by n (the length of the input). Step 3. is also doable in polynomial time (e.g. by nondeterministically inserting m separation symbols $\#$ into the input string w). In step 4. the for loop is run at most n times and every run takes at most $O(n^k)$. So the total running time is $O(n^{k+1})$. This means that M is a poly-time nondeterministic decider for L_1^* .